

ASSIGNMENT – 16.1

Name: P.Sai Bunny

Hall.No: 2303A51911

Batch – 12

Task1: Design a database schema for a Student Information System and generate queries using AI.

```
1  -- Task: Design a database schema for a Student Information System and generate queries using AI.
2  -- Drop tables if already exist (to avoid errors)
3  DROP TABLE IF EXISTS Enrollments;
4  DROP TABLE IF EXISTS Students;
5  DROP TABLE IF EXISTS Courses;
6
7  -- Students Table
8  CREATE TABLE Students (
9      student_id INTEGER PRIMARY KEY AUTOINCREMENT,
10     first_name TEXT NOT NULL,
11     last_name TEXT NOT NULL,
12     date_of_birth TEXT NOT NULL,
13     email TEXT UNIQUE NOT NULL
14 );
15
16 -- Courses Table
17 CREATE TABLE Courses (
18     course_id INTEGER PRIMARY KEY AUTOINCREMENT,
19     course_name TEXT NOT NULL,
20     course_code TEXT UNIQUE NOT NULL,
21     credits INTEGER NOT NULL
22 );
23
24 -- Enrollments Table
25 CREATE TABLE Enrollments (
26     enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
27     student_id INTEGER NOT NULL,
28     course_id INTEGER NOT NULL,
29     enrollment_date TEXT NOT NULL,
30     FOREIGN KEY (student_id) REFERENCES Students(student_id),
31     FOREIGN KEY (course_id) REFERENCES Courses(course_id)
32 );
```

1. Query : Insert a new student record

```
24 -- Enrollments Table
25 CREATE TABLE Enrollments (
26     enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
27     student_id INTEGER NOT NULL,
28     course_id INTEGER NOT NULL,
29     enrollment_date TEXT NOT NULL,
30     FOREIGN KEY (student_id) REFERENCES Students(student_id),
31     FOREIGN KEY (course_id) REFERENCES Courses(course_id)
32 );
33
34 -- Insert Students
35 INSERT INTO Students (first_name, last_name, date_of_birth, email)
36 VALUES
37 ('John', 'Doe', '2000-01-15', 'john@example.com'),
38 ('Jane', 'Smith', '1999-05-20', 'jane@example.com');
39
40 -- Insert Courses
41 INSERT INTO Courses (course_name, course_code, credits)
42 VALUES
43 ('Introduction to Computer Science', 'CS101', 3),
44 ('Database Systems', 'CS102', 4);
45
46 -- Enrollments
47 INSERT INTO Enrollments (student_id, course_id, enrollment_date)
48 VALUES
49 (1, 1, '2024-01-10'),
50 (1, 2, '2024-01-12'),
51 (2, 1, '2024-01-11');
```

2. Query : Fetch all courses enrolled by a student.

```
33 -- Insert Students
34 INSERT INTO Students (first_name, last_name, date_of_birth, email)
35 VALUES
36 ('John', 'Doe', '2000-01-15', 'john@example.com'),
37 ('Jane', 'Smith', '1999-05-20', 'jane@example.com');
38
39 -- Insert Courses
40 INSERT INTO Courses (course_name, course_code, credits)
41 VALUES
42 ('Introduction to Computer Science', 'CS101', 3),
43 ('Database Systems', 'CS102', 4);
44
45 -- Enrollments
46 INSERT INTO Enrollments (student_id, course_id, enrollment_date)
47 VALUES
48 (1, 1, '2024-01-10'),
49 (1, 2, '2024-01-12'),
50 (2, 1, '2024-01-11');
51
52 -- Query
53 SELECT s.first_name, s.last_name, c.course_name
54 FROM Students s
55 JOIN Enrollments e ON s.student_id = e.student_id
56 JOIN Courses c ON e.course_id = c.course_id
57 WHERE c.course_code = 'CS101';
```

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 2 of 2

first_name	last_name	course_name
John	Doe	Introduction to Computer Science
Jane	Smith	Introduction to Computer Science

3. Query : Count number of students in each course.

```
-- Count number of students in each course.
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id;
```

SQL ▾ < 1 / 1 > 1 - 2 of 2

course_name	student_count
Introduction to Computer Science	2
Database Systems	1

TASK 2 : Create schema and queries for a Hospital Management System.

```
-- Create schema and queries for a Hospital Management System
-- Drop tables if already exist (to avoid errors)
DROP TABLE IF EXISTS Appointments;
DROP TABLE IF EXISTS Patients;
DROP TABLE IF EXISTS Doctors;
-- Patients Table
CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    date_of_birth TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);
-- Doctors Table
CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    specialty TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);
-- Appointments Table
CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    patient_id INTEGER NOT NULL,
    doctor_id INTEGER NOT NULL,
    appointment_date TEXT NOT NULL,
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id)
);
```

```
-- Create schema and queries for a Hospital Management System
-- Drop tables if already exist (to avoid errors)
DROP TABLE IF EXISTS Appointments;
DROP TABLE IF EXISTS Patients;
DROP TABLE IF EXISTS Doctors;
-- Patients Table
CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    date_of_birth TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);
-- Doctors Table
CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    specialty TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);
-- Appointments Table
CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    patient_id INTEGER NOT NULL,
    doctor_id INTEGER NOT NULL,
    appointment_date TEXT NOT NULL,
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id)
);
```

[illegible]

Query 1: List all appointments for a specific doctor.

[illegible]

Ask for Edits Ctrl+I

SQL ▾	< 1 / 1 >	1 - 0 of 0	   
SQL ▾	< 1 / 1 >	1 - 0 of 0	   
SQL ▾	< 1 / 1 >	1 - 0 of 0	   
SQL ▾	< 1 / 1 >	1 - 0 of 0	   
SQL ▾	< 1 / 1 >	1 - 0 of 0	   
SQL ▾	< 1 / 1 >	1 - 0 of 0	   
SQL ▾	< 1 / 1 >	1 - 0 of 0	   
SQL ▾	< 1 / 1 >	1 - 2 of 2	   

first_name	last_name	appointment_date
Emily	Clark	2024-07-01
Michael	Brown	2024-07-10

Query 2 : Retrieve patient history by patient ID.

```
-- List all appointments for a specific doctor.
SELECT p.first_name, p.last_name, a.appointment_date
FROM Patients p
JOIN Appointments a ON p.patient_id = a.patient_id
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE d.email = 'sarah.johnson@example.com';
```

```
-- Retrieve patient history by patient ID.
SELECT d.first_name, d.last_name, a.appointment_date
FROM Doctors d
JOIN Appointments a ON d.doctor_id = a.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
WHERE p.patient_id = 1;
```

first_name	last_name	appointment_date
Emily	Clark	2024-07-01
Michael	Brown	2024-07-10

first_name	last_name	appointment_date
Dr. Sarah	Johnson	2024-07-01
Dr. David	Smith	2024-07-15

Query 3 : Count total patients treated by each doctor.

```
-- Count total patients treated by each doctor.
SELECT d.first_name, d.last_name, COUNT(a.patient_id) AS patient_count
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id;
```

first_name	last_name	patient_count
Dr. Sarah	Johnson	2
Dr. David	Smith	1

TASK 3 : Design schema for a Library Management System

```
-- Design schema for a Library Management System
-- Drop tables if already exist (to avoid errors)
DROP TABLE IF EXISTS Borrowings;
DROP TABLE IF EXISTS Books;
DROP TABLE IF EXISTS Members;
-- Members Table
CREATE TABLE Members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);
-- Books Table
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    isbn TEXT UNIQUE NOT NULL
);
-- Borrowings Table
CREATE TABLE Borrowings (
    borrowing_id INTEGER PRIMARY KEY AUTOINCREMENT,
    member_id INTEGER NOT NULL,
    book_id INTEGER NOT NULL,
    borrow_date TEXT NOT NULL,
    return_date TEXT,
    FOREIGN KEY (member_id) REFERENCES Members(member_id),
    FOREIGN KEY (book_id) REFERENCES Books(book_id)
);
```

```
-- Insert Members
INSERT INTO Members (first_name, last_name, email)
VALUES
('Alice', 'Johnson', 'alice@example.com'),
('Bob', 'Smith', 'bob@example.com');
-- Insert Books
INSERT INTO Books (title, author, isbn)
VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', '978-0-7432-7356-5'),
('To Kill a Mockingbird', 'Harper Lee', '978-0-06-114003-6');
-- Insert Borrowings
INSERT INTO Borrowings (member_id, book_id, borrow_date, return_date)
VALUES
(1, 1, '2024-08-01', '2024-08-15'),
(1, 2, '2024-08-16', NULL),
(2, 1, '2024-08-05', '2024-08-20');
```

```
-- Insert Members
INSERT INTO Members (first_name, last_name, email)
VALUES
('Alice', 'Johnson', 'alice@example.com'),
('Bob', 'Smith', 'bob@example.com');
-- Insert Books
INSERT INTO Books (title, author, isbn)
VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', '978-0-7432-7356-5'),
('To Kill a Mockingbird', 'Harper Lee', '978-0-06-114003-6');
-- Insert Borrowings
INSERT INTO Borrowings (member_id, book_id, borrow_date, return_date)
VALUES
(1, 1, '2024-08-01', '2024-08-15'),
(1, 2, '2024-08-16', NULL),
(2, 1, '2024-08-05', '2024-08-20');
```

Query 1 : Retrieve all books currently issued.

```
-- Retrieve all books currently issued.
SELECT b.title, m.first_name, m.last_name, br.borrow_date
FROM Books b
JOIN Borrowings br ON b.book_id = br.book_id
JOIN Members m ON br.member_id = m.member_id
WHERE br.return_date IS NULL;
```

SQL < 1 / 1 > 1 - 1 of 1

title	first_name	last_name	borrow_date
To Kill a Mockingbird	Alice	Johnson	2024-08-16

Query 2 : Find the most purchased product.

```
-- Retrieve all books currently issued.
SELECT b.title, m.first_name, m.last_name, br.borrow_date
FROM Books b
JOIN Borrowings br ON b.book_id = br.book_id
JOIN Members m ON br.member_id = m.member_id
WHERE br.return_date IS NULL;
```

```
-- Find the most purchased product.
SELECT b.title, COUNT(br.borrowing_id) AS borrow_count
FROM Books b
JOIN Borrowings br ON b.book_id = br.book_id
GROUP BY b.book_id
ORDER BY borrow_count DESC
LIMIT 1;
```

SQL < 1 / 1 > 1 - 0 of 0

SQL < 1 / 1 > 1 - 1 of 1

title	first_name	last_name	borrow_date
To Kill a Mockingbird	Alice	Johnson	2024-08-16

SQL < 1 / 1 > 1 - 1 of 1

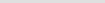
title	borrow_count
The Great Gatsby	2

Query 3 : Count number of books loaned by each member.

```
-- Find the most purchased product.
SELECT b.title, COUNT(br.borrowing_id) AS borrow_count
FROM Books b
JOIN Borrowings br ON b.book_id = br.book_id
GROUP BY b.book_id
ORDER BY borrow_count DESC
LIMIT 1;

-- Count number of books loaned by each member.
SELECT m.first_name, m.last_name, COUNT(br.borrowing_id) AS borrow_count
FROM Members m
LEFT JOIN Borrowings br ON m.member_id = br.member_id
GROUP BY m.member_id;
```

To Kill a Mockingbird	Alice	Johnson	2024-08-16
-----------------------	-------	---------	------------

SQL ▾ < 1 / 1 > 1 - 1 of 1 

title	borrow_count
The Great Gatsby	2

SQL ▾ < 1 / 1 > 1 - 2 of 2 

first_name	last_name	borrow_count
Alice	Johnson	2
Bob	Smith	1

TASK 4 : Design a database for an E-commerce platform.

```
-- Design a database for an E-commerce platform.
-- Drop tables if already exist (to avoid errors)
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Customers;
-- Customers Table
CREATE TABLE Customers (
    customer_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);
-- Products Table
CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    price REAL NOT NULL,
    stock INTEGER NOT NULL
);
-- Orders Table
CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    order_date TEXT NOT NULL,
    quantity INTEGER NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

```
-- Insert Customers
INSERT INTO Customers (first_name, last_name, email)
VALUES
('John', 'Doe', 'john@example.com'),
('Jane', 'Smith', 'jane@example.com');
-- Insert Products
INSERT INTO Products (product_name, price, stock)
VALUES
('Laptop', 999.99, 10),
('Mouse', 29.99, 50);
-- Insert Orders
INSERT INTO Orders (customer_id, product_id, order_date, quantity)
VALUES
(1, 1, '2024-08-01', 1),
(1, 2, '2024-08-01', 2),
(2, 1, '2024-08-02', 1);
```

Query 1 : Retrieve all orders by a user.

```
-- Retrieve all orders by a user.
SELECT c.first_name, c.last_name, p.product_name, o.order_date, o.quantity
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN Products p ON o.product_id = p.product_id
WHERE c.email = 'john@example.com';
```

SQL < 1 / 1 > 1 - 2 of 2

first_name	last_name	product_name	order_date	quantity
John	Doe	Laptop	2024-08-01	1
John	Doe	Mouse	2024-08-01	2

2 : Find the most purchased product. Query

```
-- Retrieve all orders by a user.
SELECT p.product_name, SUM(o.quantity) AS total_quantity
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN Products p ON o.product_id = p.product_id
WHERE c.email = 'john@example.com';

-- Find the most purchased product.
SELECT p.product_name, SUM(o.quantity) AS total_quantity
FROM Products p
JOIN Orders o ON p.product_id = o.product_id
GROUP BY p.product_id
ORDER BY total_quantity DESC
LIMIT 1;
```

SQL < 1 / 1 > 1 - 2 of 2

first_name	last_name	product_name	order_date	quantity
John	Doe	Laptop	2024-08-01	1
John	Doe	Mouse	2024-08-01	2

SQL < 1 / 1 > 1 - 1 of 1

product_name	total_quantity
Laptop	2

Query 3 : Calculate total revenue in a given month

```

-- Find the most purchased product.
SELECT p.product_name, SUM(o.quantity) AS total_quantity
FROM Products p
JOIN Orders o ON p.product_id = o.product_id
GROUP BY p.product_id
ORDER BY total_quantity DESC

-- Calculate total revenue in a given month
SELECT strftime('%Y-%m', o.order_date) AS month, SUM(p.price * o.quantity) AS total_revenue
FROM Orders o
JOIN Products p ON o.product_id = p.product_id
GROUP BY month
ORDER BY month;

```

```

-- Find the most purchased product.
SELECT p.product_name, SUM(o.quantity) AS total_quantity
FROM Products p
JOIN Orders o ON p.product_id = o.product_id
GROUP BY p.product_id
ORDER BY total_quantity DESC

-- Calculate total revenue in a given month
SELECT strftime('%Y-%m', o.order_date) AS month, SUM(p.price * o.quantity) AS total_revenue
FROM Orders o
JOIN Products p ON o.product_id = p.product_id
GROUP BY month
ORDER BY month;

```

John	Doe	Mouse	2024-08-01	2
------	-----	-------	------------	---

SQL ▼ < 1 / 1 > 1 - 1 of 1 🗑️ ↩️ ⏏️ 🔍

product_name	total_quantity
Laptop	2

SQL ▼ < 1 / 1 > 1 - 1 of 1 🗑️ ↩️ ⏏️ 🔍

month	total_revenue
2024-08	2059.96

TASK 5 : Design a database schema for a University Examination System and generate SQL queries using AI.

```

-- Design a database schema for a University Examination System and generate SQL queries using AI.
-- Drop tables if already exist (to avoid errors)
DROP TABLE IF EXISTS ExamResults;
DROP TABLE IF EXISTS Students;
DROP TABLE IF EXISTS Exams;
-- Students Table
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    date_of_birth TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);
-- Exams Table
CREATE TABLE Exams (
    exam_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL,
    exam_date TEXT NOT NULL
);
-- ExamResults Table
CREATE TABLE ExamResults (
    result_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    exam_id INTEGER NOT NULL,
    score REAL NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
);

```

```

-- Exams Table
CREATE TABLE Exams (
    exam_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL,
    exam_date TEXT NOT NULL
);
-- ExamResults Table
CREATE TABLE ExamResults (
    result_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    exam_id INTEGER NOT NULL,
    score REAL NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
);
-- Insert Students
INSERT INTO Students (first_name, last_name, date_of_birth, email)
VALUES
('Alice', 'Johnson', '2001-03-22', 'alice@example.com'),
('Bob', 'Smith', '2000-07-15', 'bob@example.com');
-- Insert Exams
INSERT INTO Exams (course_name, exam_date)
VALUES
('Mathematics', '2024-08-15'),
('Physics', '2024-08-20');
-- Insert Exam Results
INSERT INTO ExamResults (student_id, exam_id, score)
VALUES
(1, 1, 85.5),
(1, 2, 92.0),
(2, 1, 78.0),
(2, 2, 88.5);

```

Query 1 : Insert examination results for a student.

```
-- Query 1 : Insert examination results for a student.
INSERT INTO ExamResults (student_id, exam_id, score)
VALUES (1, 1, 85.5);
```

Query 2: Retrieve subject-wise marks for a student.

```
-- Query 2: Retrieve subject-wise marks for a student.
SELECT e.course_name, er.score
FROM ExamResults er
JOIN Exams e ON er.exam_id = e.exam_id
WHERE er.student_id = 1;
```

course_name	score
Mathematics	85.5
Physics	92.0
Mathematics	85.5

Query 3 : Calculate average marks for each subject.

```
-- Query 3: Calculate the average score for a specific exam.
SELECT e.course_name, AVG(er.score) AS average_score
FROM ExamResults er
JOIN Exams e ON er.exam_id = e.exam_id
WHERE e.course_name = 'Mathematics'
GROUP BY e.exam_id;
```

Mathematics	85.5
-------------	------

course_name	average_score
Mathematics	83.0

LAB OBSERVATIONS

◇ Objective

To design and implement database schemas for various real-world systems and perform SQL operations such as table creation, data insertion, and query execution using SQL.

◇ Observation 1: Student Information System

- Created three tables: **Students, Courses, Enrollments**.
- Established **many-to-many relationship** using a bridge table (Enrollments).
- Used **primary keys** and **foreign keys** to maintain data integrity.
- Inserted sample student and course data successfully.
- Performed queries to:
 - Retrieve students enrolled in a specific course.
 - Count number of students in each course.
- Observed that **JOIN operations** help combine data from multiple tables.

◇ Observation 2: Hospital Management System

- Designed tables: **Patients, Doctors, Appointments**.
- Implemented relationships using foreign keys.
- Inserted patient, doctor, and appointment data.

- Executed queries to:
 - List appointments for a specific doctor.
 - Retrieve patient history.
 - Count total patients handled by each doctor.
- Observed that **LEFT JOIN** helps include doctors with no appointments.

◇ **Observation 3: Library Management System**

- Created tables: **Members, Books, Borrowings**.
- Maintained relationships using foreign keys.
- Inserted borrowing records with return status.
- Executed queries to:
 - Find currently issued books (using NULL condition).
 - Identify most borrowed book.
 - Count books borrowed by each member.
- Observed that **NULL values** are useful for tracking incomplete actions (e.g., not returned books).

◇ **Observation 4: E-Commerce System**

- Designed tables: **Customers, Products, Orders**.
- Established relationships using foreign keys.
- Inserted sample order and product data.
- Executed queries to:
 - Retrieve all orders by a specific customer.
 - Find most purchased product.
 - Calculate total revenue per month.
- Observed that **aggregate functions (SUM, COUNT)** are useful for business analysis.

◇ Observation 5: University Examination System

- Created tables: **Students**, **Exams**, **ExamResults**.
- Linked students and exams using a result table.
- Inserted exam and result data.
- Executed queries to:
 - Retrieve subject-wise marks.
 - Calculate average scores for exams.
- Observed that **GROUP BY** and **AVG functions** help in performance evaluation.